COMSATS University Islamabad, Lahore Campus

Operating System Lab Assignment 02

Submitted to: Ma'am Zeenat Afzal

Submitted by:

Afrasayyab Munawar SP24-BSE-007

Samiullah Ashfaq SP24-BSE-107

Section: A

Question 01: Array and Two Child Processes

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <stdbool.h>

// Helper function to check if a number is prime
bool is_prime(int num) {
    if (num <= 1) return false;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return false;
    }
    return true;
}

int main() {
    int arr[1000];

    // Seed the random number generator
    srand(time(NULL));

    // Populate the array with random integers between 1 and 100
    for (int i = 0; i < 1000; i++) {
        arr[i] = (rand() % 100) + 1;
    }

    // Create the first child process
    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("First fork failed");
        exit(1);
    } else if (pid1 == 0) {
        // --- FIRST CHILD PROCESS ---
```

```
        int prime_count = 0;
        for (int i = 0; i < 500; i++) {
            if (is_prime(arr[i])) {
                prime_count++;
            }
        }
        printf("First Child (PID: %d): Found %d prime numbers in the first 500 elements.\n", getpid(),
prime_count);
        exit(0); // Exit so child doesn't execute parent code
    } else {
        // --- PARENT PROCESS ---
        // Create the second child process
        pid_t pid2 = fork();

        if (pid2 < 0) {
            perror("Second fork failed");
            exit(1);
        } else if (pid2 == 0) {
            // --- SECOND CHILD PROCESS ---
            int prime_count = 0;
            for (int i = 500; i < 1000; i++) {
                if (is_prime(arr[i])) {
                    prime_count++;
                }
            }
            printf("Second Child (PID: %d): Found %d prime numbers in the remaining 500 elements.
\n", getpid(), prime_count);
            exit(0); // Exit so child doesn't execute parent code
        } else {
            // Parent must wait for both children to terminate to prevent zombie processes
            wait(NULL);
            wait(NULL);
            printf("Parent (PID: %d): Both child processes have completed successfully.\n", getpid());
        }
    }

    return 0;
}
```

Output:

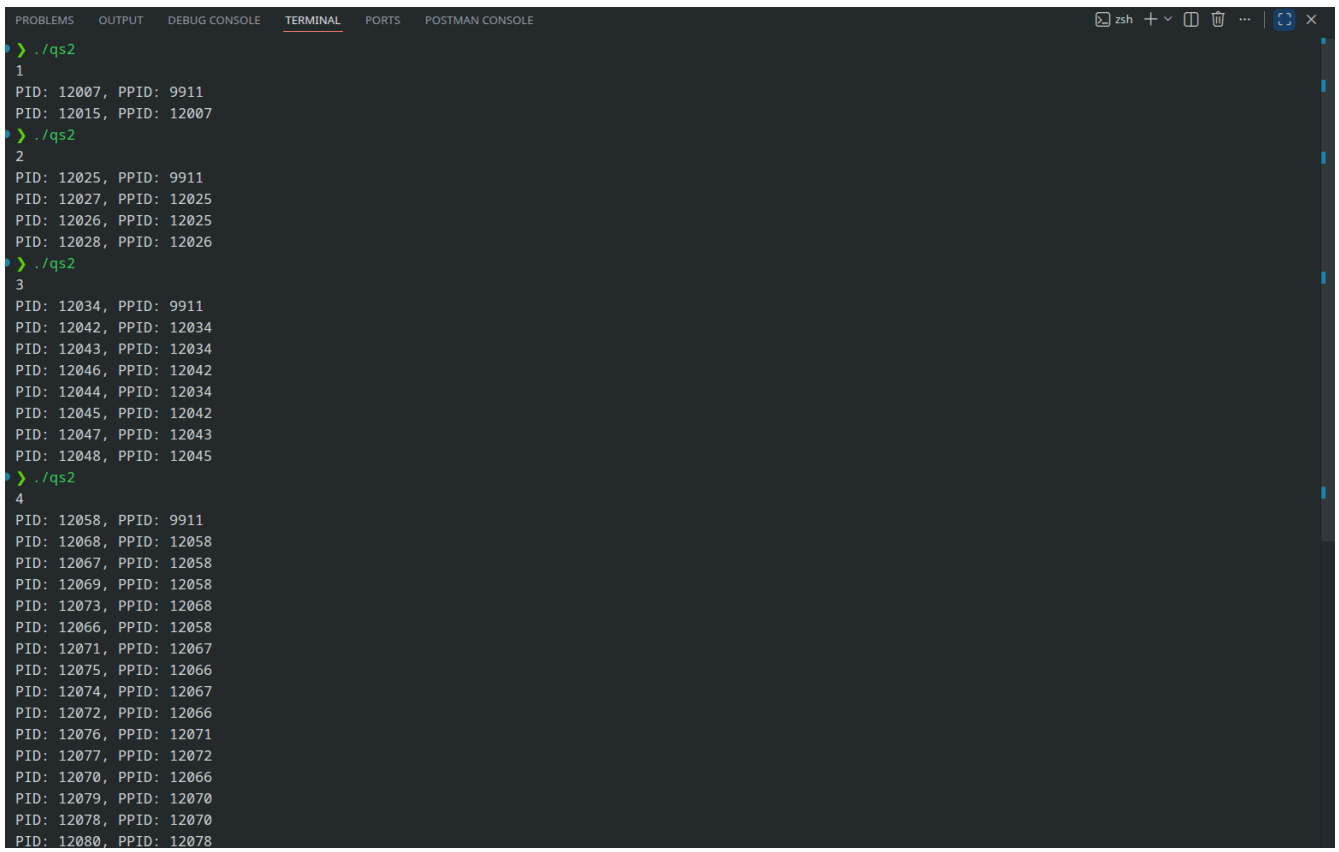
```
❯ gcc qs1.c -o qs1
❯ ./qs1
First Child (PID: 10108): Found 120 prime numbers in the first 500 elements.
Second Child (PID: 10109): Found 137 prime numbers in the remaining 500 elements.
Parent (PID: 10107): Both child processes have completed successfully.
❯ ./qs1
First Child (PID: 10144): Found 139 prime numbers in the first 500 elements.
Second Child (PID: 10145): Found 125 prime numbers in the remaining 500 elements.
Parent (PID: 10143): Both child processes have completed successfully.
❯ ./qs1
First Child (PID: 10176): Found 117 prime numbers in the first 500 elements.
Second Child (PID: 10177): Found 115 prime numbers in the remaining 500 elements.
Parent (PID: 10175): Both child processes have completed successfully.
❯ ./qs1
First Child (PID: 10189): Found 123 prime numbers in the first 500 elements.
Second Child (PID: 10190): Found 125 prime numbers in the remaining 500 elements.
Parent (PID: 10188): Both child processes have completed successfully.
❯ ./qs1
First Child (PID: 10206): Found 158 prime numbers in the first 500 elements.
Second Child (PID: 10207): Found 143 prime numbers in the remaining 500 elements.
Parent (PID: 10205): Both child processes have completed successfully.

~/Documents/Code/OS_Code
❯
```

Question 02: Generic n Forks

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n;

    // Read 'n' silently because the question specifies "no other print statement"
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    // Generic loop for n forks
    for (int i = 0; i < n; i++) {
        if (fork() < 0) {
            // In case fork fails, terminate to avoid broken process trees
            exit(1);
        }
    }
```

```
    // Print PID and PPID. This will be executed by 2^n processes.
    printf("PID: %d, PPID: %d\n", getpid(), getppid());

    // Wait for all child processes to ensure no process becomes an orphan or zombie.
    // wait(NULL) returns -1 when a process has no more children left to wait for.
    while (wait(NULL) > 0);

    return 0;
}
```

Output:



```
❯ ./qs2
5
PID: 12096, PPID: 9911
PID: 12104, PPID: 12096
PID: 12103, PPID: 12096
PID: 12102, PPID: 12096
PID: 12109, PPID: 12103
PID: 12107, PPID: 12102
PID: 12115, PPID: 12107
PID: 12108, PPID: 12100
PID: 12101, PPID: 12096
PID: 12127, PPID: 12120
PID: 12116, PPID: 12105
PID: 12100, PPID: 12096
PID: 12129, PPID: 12116
PID: 12106, PPID: 12101
PID: 12118, PPID: 12108
PID: 12119, PPID: 12100
PID: 12112, PPID: 12102
PID: 12128, PPID: 12113
PID: 12105, PPID: 12100
PID: 12130, PPID: 12118
PID: 12117, PPID: 12101
PID: 12110, PPID: 12105
PID: 12111, PPID: 12101
PID: 12114, PPID: 12106
PID: 12123, PPID: 12108
PID: 12120, PPID: 12110
PID: 12126, PPID: 12114
PID: 12121, PPID: 12106
PID: 12113, PPID: 12100
PID: 12125, PPID: 12111
PID: 12124, PPID: 12110
PID: 12122, PPID: 12105

~/Documents/Code/OS_Code
❯
```

When you place a fork() system call inside a loop without explicitly telling the child process to exit(), the number of processes grows exponentially. This happens because fork() creates an exact duplicate of the calling process, and crucially, *both* the original parent and the newly created child continue executing the loop from that exact spot. So, during the first iteration, the parent creates one child, resulting in 2 total processes. In the second iteration, both of those processes execute fork(), doubling the total to 4. In the third iteration, all 4 processes call fork(), making it 8, and so on. Every time the loop runs, the total number of active processes doubles. This creates a massive, branching hierarchy of processes, ultimately resulting in 2^n total processes (where n is the number of loop iterations).